

ZFS

Sistema de Archivos Avanzado
para la Gestión Segura y
Escalable del Almacenamiento

Brais Loeda Estremado

Índice

1. Introducción al proyecto	4
2. Qué es ZFS y por qué se usa	5
3. Por qué se eligió este proyecto:	8
4. Topología y direccionamiento del laboratorio	8
5. Fase 0 — Preparación del laboratorio en VirtualBox	9
6. Fase 1 — Instalación de ZFS sobre Debian 13	10
7. Fase 2 — Crear el pool, los datasets y el zvol	12
8. Fase 3 - Configurar los servicios	15
9. Fase 4 — Snapshots automáticos y replicación (SSH)	19
10. Fase 5 — Pruebas y simulación de fallos	22
11. Uso de contraseñas y usuarios	25
12. Presupuesto	26
13. Mejoras a futuro	28
14. Conclusiones y dificultades encontradas	29
15. Scripts de configuración	30
16. Bibliografía	38

1. Introducción al proyecto

El objetivo de este proyecto es montar un servidor de almacenamiento basado en ZFS que comparta ficheros por NFS y por Samba y que, además, exponga un disco por red mediante iSCSI; todo ello sobre el mismo almacenamiento y protegido con snapshots automáticos y con una réplica completa en un segundo servidor. La idea es demostrar, sobre un laboratorio virtualizado, que con una sola tecnología de almacenamiento se pueden cubrir los tres tipos de acceso más habituales y, encima, ganar en seguridad e integridad de los datos.

La memoria no se limita a poner comandos: para cada uno se explica qué hace, por qué se hace así y cómo se comprueba que ha funcionado, de manera que cualquiera que acabe de terminar el ciclo pueda reproducir el montaje sin atascarse. Se asume el manejo básico de Linux (terminal, apt, montar dispositivos) y la explicación se centra en lo que es nuevo: ZFS, los tres protocolos y la replicación.

El proyecto se divide en seis fases con un orden lógico: primero se prepara el terreno, después se construye el almacenamiento, luego se ponen los servicios encima, se protegen los datos y por último se prueba todo, incluyendo la simulación de fallos.

2. Qué es ZFS y por qué se usa

Conviene empezar por los conceptos, porque sin entenderlos los comandos se quedan en un «copia y pega» que no sirve para defender el proyecto. ZFS es a la vez un gestor de volúmenes y un sistema de ficheros, y esa unión es justo lo que le da sus ventajas.

2.1. El modelo en capas

En un sistema clásico hay dos piezas separadas: por un lado lo que gestiona los discos (un gestor de volúmenes tipo LVM, o una controladora RAID) y por otro el sistema de ficheros (ext4, NTFS...) que organiza los datos por encima. ZFS junta las dos cosas en una sola pieza, y ahí está la clave: como sabe a la vez cómo están los discos y cómo están los datos, puede proteger la información de un modo que un ext4 normal no puede.

La estructura, de abajo a arriba, es la siguiente:

- Discos físicos. Los discos reales; en este caso, discos virtuales de VirtualBox.
- vdev. Un grupo de discos que ZFS trata como una unidad y donde se define la redundancia (en este proyecto, un espejo de dos discos).
- zpool. La suma de los vdevs: la capacidad total y donde se aplican opciones globales como la compresión.
- Datasets y zvols. Dentro del pool se crea lo que se va a usar: datasets (sistemas de ficheros, para NFS y Samba) y zvols (discos en bruto, para iSCSI).

Capa	Qué es	En este proyecto
Servicios	NFS / Samba / iSCSI	comparten datasets o el zvol
Datasets / zvol	ficheros o bloque	tank/nfs, tank/samba, tank/iscsi-lun0
zpool	capacidad total	pool «tank» (y «backup» en la réplica)
vdev	redundancia	mirror de 2 discos
Discos físicos	el hardware	discos SATA de VirtualBox

2.2. Copy-on-write, integridad y snapshot.

El truco principal de ZFS se llama copy-on-write («copiar al escribir»). Cuando se modifica un bloque, ZFS no lo sobrescribe en el mismo sitio: escribe los datos nuevos en otro lugar libre y solo al final mueve el «puntero» para que apunte a los nuevos. Los bloques viejos no se tocan hasta que ya no hacen falta. Por eso un snapshot puede seguir apuntando al bloque viejo mientras el estado actual usa el nuevo, y los bloques que no han cambiado se comparten, de modo que una «foto» apenas ocupa al principio.

Esto da tres ventajas que se aprovechan en el proyecto:

- No hay corrupción si se va la luz: los datos viejos siguen intactos hasta que los nuevos están realmente guardados.
- Detecta y repara errores gracias a una suma de verificación (checksum) por bloque, que incluso se puede forzar con `zpool scrub`.
- Los snapshots son casi gratis, lo que hace viable tenerlos automáticos y replicar sólo las diferencias.

ZFS usa además la RAM como caché de lectura (el ARC); por eso en este montaje se da más memoria a los servidores que a los clientes.

2.3. Redundancia: por qué se usa mirror

La redundancia se decide al crear el vdev y determina cuántos discos pueden estropearse sin perder datos. Para este proyecto se eligió mirror (espejo) en ambos servidores. En un mirror se escribe lo mismo en dos discos a la vez; si uno falla, el otro tiene todo. Es rápido, fácil de explicar y la reconstrucción (llamada resilver) es ágil.

El coste es que se usan dos discos para la capacidad de uno, pero para el laboratorio compensa porque la demostración «se quita un disco y el servidor sigue funcionando» queda clarísima. La alternativa habría sido RAID-Z1 (paridad en tres o más discos, más espacio aprovechado pero resilver más lento); se descartó por simplicidad.

2.4. Acceso a ficheros frente a acceso a bloque

Esta es la distinción que más cuesta al principio y la que explica por qué se usan datasets para unos servicios y un zvol para otro:

- **Acceso a ficheros (NFS y Samba)**

El servidor es el dueño del sistema de ficheros (un dataset ya formateado) y comparte carpetas y ficheros. Funciona como compartir una carpeta en red: varios usuarios a la vez y los permisos se controlan en el servidor.

- **Acceso a bloque (iSCSI)**

El servidor entrega al cliente un disco en bruto por la red (un zvol), como si le conectara un disco nuevo. Es el cliente quien lo formatea (ext4, NTFS...) y lo usa como un disco local. Normalmente lo utiliza un único cliente.

3. Por qué se eligió este proyecto:

Se eligió este proyecto porque es algo real que usa cualquier empresa: tener un servidor donde guardar ficheros, compartirlos con diferentes sistemas operativos (Windows y Linux) y tener una copia de seguridad automática por si algo falla.

Además, permite aprender y practicar varias cosas importantes para un administrador de sistemas: gestionar almacenamiento, automatizar tareas con scripts, replicar datos entre servidores, y usar SSH para conexiones seguras.

El proyecto es totalmente viable con software libre y VirtualBox, lo que facilita hacerlo en el laboratorio sin costes. Y lo mejor es que los resultados se ven rápido: ficheros compartidos, datos copiados automáticamente, discos nuevos conectados por red.

Es el tipo de infraestructura que existe en casi cualquier pequeña empresa, así que después del ciclo se pueden aplicar estos conocimientos directamente en un trabajo.

4. Topología y direccionamiento del laboratorio

Se montan cuatro máquinas virtuales con dos adaptadores de red cada una: uno en NAT (solo para internet) y otro en una red interna privada (192.168.100.0/24) por la que se hablan entre ellas con IPs fijas. En esta red se deshabilita el DHCP y se asignan las direcciones a mano.

Máquina	Rol	IP (red privada)	Qué se instala
zfs-primary	Servidor primario	192.168.100.10	zfs (mirror), nfs-kernel-server, samba, targetcli-fb
zfs-replica	Secundario (réplica)	192.168.100.11	zfs (mirror) que recibe la réplica
cliente-linux	Cliente de pruebas	192.168.100.12	nfs-common, cifs-utils, open-iscsi
cliente-windows	Cliente opcional	192.168.100.13	Explorador (Samba) + iniciador iSCSI de Windows

5. Fase 0 — Preparación del laboratorio en VirtualBox

Hay dos ideas que conviene explicar bien porque condicionan el resto del proyecto:

- **Por qué dos redes.**

El adaptador NAT permite descargar paquetes, pero las máquinas en NAT no se ven bien entre sí. Por eso se añade un segundo adaptador en una red interna privada, donde se les pone una IP fija. Así el tráfico de NFS, Samba, iSCSI y la réplica viaja por una red controlada.

- **Por qué discos aparte.**

ZFS está pensado para gestionar discos enteros. Si se metiera el pool en el disco del sistema, no se podría simular un fallo sin cargarse el arranque. Por eso a `zfs-primary` y a `zfs-replica` se les añaden dos discos virtuales solo para ZFS (el mirror), aparte del disco del SO, en controladora SATA.

5.1. Máquinas base y características

Se crea primero una máquina con Debian 13 mínima, sin entorno gráfico, que hará de servidor principal: 4 GB de RAM y un disco de 20 GB para el sistema operativo.

La réplica tiene que tener exactamente las mismas características, así que se puede clonar (cambiando nombre de máquina y regenerando la MAC) o crear de nuevo igual que la primaria. Los dos clientes necesitan: 8GB de RAM y 120GB de disco para el cliente-windows y 2GB de RAM y 10 GB de disco para el cliente-linux, que hará falta más adelante para probar Samba e iSCSI desde Windows.

Máquina	RAM	Disco SO	Discos ZFS
<code>zfs-primary</code>	4 GB	20 GB	2 × 10 GB (SATA)
<code>zfs-replica</code>	4 GB	20 GB	2 × 10 GB (SATA)
<code>cliente-linux</code>	2 GB	10 GB	—
<code>cliente-windows</code>	8 GB	120 GB	—

5.2. Pasos en la interfaz de VirtualBox

1. Se crea una VM con Debian 13, se instala el sistema, se actualiza y se añaden `sudo` y `openssh-server`.
2. Se apaga la VM y se clona (clon enlazado) para tener la réplica y el cliente Linux; al clonar se marca “generar nuevas MAC”.
3. A `zfs-primary` y a `zfs-replica` se les añaden los dos discos extra para el mirror, en controladora SATA. No se formatean ni se les da ninguna propiedad: de eso se encarga ZFS.
4. Se pone el segundo adaptador en «Red interna» con el mismo nombre (por ejemplo *lan-zfs*) y se asignan las IPs fijas.

5.3. Discos para ZFS

A los dos servidores se les añaden dos discos virtuales adicionales de 10 GB cada uno (además del disco del SO), en controladores SATA. No se formatean ni se les da ninguna propiedad, para eso se usará ZFS.

6. Fase 1 — Instalación de ZFS sobre Debian 13

Ya que ZFS no forma parte del kernel de Linux por motivos de licencia, en Debian se instala como módulo compilado mediante DKMS. Es el sistema que recompila ese módulo automáticamente cada vez que cambia el núcleo, para que ZFS siga funcionando después de una actualización.

6.1. Habilitar contrib en repositorios.

El paquete de ZFS está en el componente “contrib” de los repositorios de Debian, que no viene activado por defecto. Edito el archivo de configuración de repositorios de Debian “`/etc/apt/sources.list`” y añado el apartado de, “contrib” tanto en `zfs-primary` como en `zfs-replica`.

6.2. Instalar e cargar ZFS

Una vez editados los repositorios recargo la lista de paquetes “`apt update`” y tengo que poner las cabeceras del núcleo que hacen falta para compilar el módulo, “`apt install -y linux-headers-amd64 dpkg-dev`”.

Lo siguiente son los tres paquetes principales, “`zfsutils-linux`” que trae los comandos `zpool`, `zfs`. El siguiente es, `zfs-dkms` es el que compila el módulo del núcleo (por eso la instalación puede tardar algo más), y por último, “`zfs-zed`”, que es demo que vigila el pool y avisa, por ejemplo, si un disco falla. Cuando toda la instalación termina lo compruebo con, “`modprobe zfs`” que carga el módulo en el núcleo actual y “`zfs version`” que imprime la versión actual de `zfs`.

Y para que `zfs` importe y monte el pool automáticamente cada vez que arranca la máquina,

```
systemctl enable --now zfs-import-cache zfs-mount zfs-zed.service zfs.target
```

Problema: El módulo se recompila contra el núcleo. Hay que mantener `linux-headers-amd64` y el núcleo sincronizados, por eso tras cada actualización del núcleo, hay que reiniciar y verificar la versión de `zfs` antes de seguir.

6.3. Saber que discos tengo

Antes de crear el pool identifiqué los discos. Listo lo que hay y después busco su identificador estable, que es mucho mejor que guiarme por “/dev/sda”, “/dev/sdb” porque esos nombres pueden cambiar de orden entre los arranques.

Primero identifico el dispositivo por sdb, sdc, etc... con `lsblk -o NAME,SIZE,TYPE,MODEL` y después lo asigno con su identificador estable `ls -l /dev/disk/by-id/...` En virtualbox ej: `ataVBOX_HARDDISK_VB...`

sdb 10GB → VBcefb9d49-8c2a3c7f

sdc 10GB → VB8b0a245b-f387ff60

7. Fase 2 — Crear el pool, los datasets y el zvol

Qué se consigue en esta fase:

Un pool tank en mirror en zfs-primary , otro pool backup en mirror en zfs2 , y dentro de tank los datasets y el zvol que después usarán los servicios.

7.1. Crear el pool en zfs-primary (Mirror).

Para crear el pool en zfs-primary se usa el siguiente comando,

```
zpool create -o ashift=12 -O compression=lz4 -O atime=off -O xattr=sa -O
acltype=posixacl tank mirror /dev/disk/by-id/ata-VBOX_HARDDISK_VBaaaaaaaa \
/dev/disk/by-id/ata-VBOX_HARDDISK_VBbbbbbbbb
```

Ya que tiene mucha información se desglosa así:

`zpool create ... tank` : crea el pool llamado tank.

`mirror disco1 disco2` : el vdev es un espejo de los dos discos (la redundancia elegida).

`-o ashift=12` : fija el tamaño de sector en 4 KiB. Es propiedad del pool y no se puede cambiar después, por eso se pone desde el principio.

`-O ...` (en mayúscula): propiedades de los datasets que se heredan: compresión lz4, atime apagado y los atributos/ACL para Samba.

Y para comprobarlo una vez terminado, `zpool status tank` debe mostrar el pool ONLINE con los dos discos en el espejo (en mirror-0), y `zpool list` la capacidad (se verá aproximadamente el tamaño de un solo disco, porque el otro es la copia).

7.2 Crear el pool en zfs2 (También mirror)

En el segundo servidor se hace exactamente lo mismo, pero el pool se llama backup porque será donde se reciba la réplica:

```
zpool create -o ashift=12 -O compression=lz4 -O atime=off backup mirror \
/dev/disk/by-id/disco1_de_zfs2 \ /dev/disk/by-id/disco2_de_zfs2
```

Para comprobarlo, igual que antes, con `zpool status backup`, y con `zpool list` se verá la capacidad.

7.3. Crear los datasets (para ficheros) en zfs-primary

Para crear los dataset para los ficheros usamos los siguientes comandos:

`zfs create tank/nfs`, aquí se guarda lo que se exportará por NFS y, `zfs create tank/samba` donde se guardará lo que se compartirá por Samba, para listarlos se usa `zfs list`, estos se montan por defecto en `/tank/nfs` y `/tank/samba`. Y para asegurarnos que ninguno de los datasets se pisan entre ellos, usamos una cuota, “`zfs set quota=3G tank/nfs`” y “`zfs set quota=3G tank/samba`”.

7.4. Crear el zvol (para bloque) en zfs-primary

Es un disco virtual de 3 GB que después se exporta por iSCSI, para crearlo se hace así:

```
zfs create -V 3G -o volblocksize=16k tank/iscsi-lun0
ls -l /dev/zvol/tank/
```

Este aparece como un dispositivo de bloque “`/dev/zvol/tank/iscsi-lun0`”.

A diferencia de un dataset, el zvol no es una carpeta sino un bloque en bruto. Con `-v 3G` se le da el tamaño y con `volblocksize=16k` el bloque interno. Conviene fijarse en la ruta `/dev/zvol/...`, que es la que después se le da a iSCSI.

7. 5 Probar el funcionamiento de las snapshots a mano

Probamos con un archivo creándolo, echo “hola” > /tank/nfs/prueba.txt, podemos tomar una “foto” con, zfs snapshot tank/nfs@antes, se borra rm /tank/nfs/prueba.txt procedemos a hacer un rollback de este archivo eliminado para recuperarlo, zfs rollback tank/nfs@antes. Y verificamos que ha vuelto el archivo.

8. Fase 3 - Configurar los servicios

Que se consigue en esta fase:

Los tres servicios funcionando en zfs-primary: NFS y Samba compartiendo datasets, e iSCSI publicando el zvol. En el capítulo 12 se da un script para cada uno. Aquí se explica qué hace cada paso. Se hace en el orden: NFS → SAMBA → iSCSI, de más sencillo a más complejo.

8.1. NFS

NFS es la forma de toda la vida de compartir carpetas entre sistemas Linux. El servidor sigue siendo el dueño del sistema de ficheros y los clientes lo montan por la red.

```
# [zfs-primary]
apt install -y nfs-kernel-server
echo ' /tank/nfs 192.168.100.0/24(rw,sync,no_subtree_check,root_squash)' >>
/etc/exports
exportfs -ra # aplica los cambios de /etc/exports
systemctl enable --now nfs-kernel-server
showmount -e localhost # muestra que se está exportando /tank/nfs
```

En vez de usar los pasos de instalación, se usaron los scripts aportados en [GitHub](#)

La línea de /etc/exports tiene su miga, así que se explica cada parte:

- **/tank/nfs**: la carpeta (el dataset) que se comparte.
- **192.168.100.0/24**: quién puede montarla; aquí, la red interna.
- **rw**: lectura y escritura.
- **sync**: confirma las escrituras cuando están realmente guardadas (más seguro).

- **no_subtree_check**: desactiva una comprobación antigua que da problemas; es lo recomendado.
- **root_squash**: si un cliente entra como root, en el servidor se trata como usuario sin privilegios. Es una medida de seguridad: que un root remoto no sea root en el servidor.

Para comprobarlo se hace lo siguiente, desde el cliente, `showmount -e zfs-primary` debe listar `/tank/nfs`. Si no, se revisa el servicio con `systemctl status nfs-kernel-server`.

8.2. Samba

Samba hace que un Linux hable el protocolo de Windows. A diferencia de NFS, exige usuario y contraseña, y sirve igual para clientes Windows y Linux.

```
# [zfs-primary]
apt install -y samba
groupadd sambausers # grupo con acceso
useradd -M -s /usr/sbin/nologin -G sambausers smbuser # usuario solo para Samba
smbpasswd -a smbuser # crea la contraseña EN LA
BASE DE DATOS de Samba
smbpasswd -e smbuser # habilita la cuenta
chgrp -R sambausers /tank/samba # la carpeta pertenece al grupo
chmod 2770 /tank/samba # permisos del grupo + bit
setgid
```

En vez de usar los pasos de instalación, se usaron los scripts aportados en [GitHub](#)

Dos cosas importantes: la contraseña de Samba es independiente de la del sistema (se guarda en una base de datos propia, por eso se usa `smbpasswd`), y el 2770 lleva un 2 delante que es el bit `setgid`: hace que todo lo que se cree dentro herede el grupo `sambausers`, para que los permisos no se descontrolen.

El recurso se añade al final de `/etc/samba/smb.conf`:

```
[global]
    workgroup = WORKGROUP
    server string = Servidor ZFS (Samba)
    security = user          # exige usuario y contraseña
    map to guest = never
    server min protocol = SMB2

[datos]                  # nombre del recurso que verán los clientes
    comment = Compartición sobre ZFS
    path = /tank/samba     # carpeta real compartida
    browseable = yes
    read only = no         # permite escribir
    guest ok = no          # nada de invitados
    valid users = @smbausers # solo los del grupo
    force group = sambausers
    create mask = 0660
    directory mask = 2770

testparm                # comprueba errores de sintaxis en el smb.conf

systemctl restart smbd nmbd
systemctl enable smbd nmbd
```

8.3. iSCSI — publicar el zvol como disco de red

iSCSI es la parte más nueva, así que conviene tener claro el vocabulario antes de empezar:

- Target: el servidor que ofrece el disco (aquí, zfs-primary).
- Iniciador: el cliente que se conecta (aquí, cliente-linux).
- IQN: el «nombre» único de cada uno (iqn.año-mes.dominio-al-revés:etiqueta).
- LUN: el número que identifica el disco concreto que se ofrece (el zvol).
- Backstore: lo que hay detrás del LUN; el zvol /dev/zvol/tank/iscsi-lun0.
- Portal: la IP y el puerto (3260) por los que escucha el target.

Se configura con targetcli, en comandos de una línea para que quede documentado y sea repetible:

```
# [zfs-primary]
apt install -y targetcli-fb
targetcli /backstores/block create name=lun0 dev=/dev/zvol/tank/iscsi-lun0
```

1) el disco (zvol)

```
targetcli /iscsi create iqn.2026-01.local.zfs-primary:target0
```

2) el target y su IQN

```
targetcli /iscsi/iqn.2026-01.local.zfs-primary:target0/tpg1/luns \
    create /backstores/block/lun0
```

3) se añade el LUN

```
targetcli /iscsi/iqn.2026-01.local.zfs-primary:target0/tpg1/acls \
    create iqn.2026-01.local.cliente-linux:init0
```

4) solo este cliente entra

```
targetcli /iscsi/iqn.2026-01.local.zfs-primary:target0/tpg1/portals \
    delete 0.0.0.0 3260
```

5) se quita el portal abierto a todos

```
targetcli /iscsi/iqn.2026-01.local.zfs-primary:target0/tpg1/portals \
    create 192.168.100.10
```

6) se escucha solo en la red interna

```
targetcli saveconfig
```

7) se guarda la configuración

```
systemctl enable --now rtlib-fb-targetctl.service
```

8) se restaura al arrancar

En vez de usar los pasos de instalación, se usaron los scripts aportados en [GitHub](#)

El paso 4 (la ACL) es clave: solo se deja entrar al iniciador cuyo IQN coincide con el que se pondrá en el cliente, así que ese IQN debe ser idéntico en las dos máquinas. Los pasos 5 y 6 cambian de “se escucha en todas las interfaces” a “se escucha solo en la red interna”.

9. Fase 4 — Snapshots automáticos y replicación (SSH)

Que se consigue en esta fase:

Dos capas de protección: snapshots automáticos locales para poder recuperar datos borrados o errores del día a día, y una réplica completa en zfs-replica para el caso de que el servidor primario falle por completo. Todo se hace con herramientas nativas de ZFS, cron y SSH, sin instalar nada extra.

9.1. Snapshots automáticos con cron

ZFS permite sacar fotos del estado de un dataset en cualquier momento. Estas fotos apenas ocupan espacio porque, gracias al copy-on-write, solo guardan los bloques que han cambiado desde el snapshot anterior. Para automatizarlas se usa cron, el planificador de tareas de Linux.

Se crea el script `zfs-snapshot.sh`, que toma un snapshot de cada dataset con la fecha y la hora como nombre, y se programa para ejecutarse cada día a las 00:00:

```
0 0 * * * root /usr/local/bin/zfs-snapshot.sh
```

Para no acumular snapshots indefinidamente, el script `zfs-cleanup.sh` conserva solo los últimos 7 y borra el resto, ejecutándose 5 minutos después:

```
5 0 * * * root /usr/local/bin/zfs-cleanup.sh
```

Para comprobarlo:

`zfs list -t snapshot` debe mostrar los snapshots de los tres datasets con la fecha del día.

9.2. Preparar SSH para la replicación

La réplica viaja cifrada por SSH entre zfs-primary y zfs-replica. Para no usar root se crea un usuario repl al que se le dan solo los permisos de ZFS necesarios mediante la delegación de privilegios propia de ZFS, que es una buena práctica de seguridad. En ambas máquinas se crea el usuario:

En vez de usar los pasos de instalación, se usaron los scripts aportados en [GitHub](#)

en AMBAS máquinas

```
useradd -m -s /bin/bash repl
```

En zfs-primary se le da permiso para enviar y en zfs-replica para recibir:

[zfs-primary] se le permite ENVIAR

```
zfs allow repl send,snapshot,hold tank
```

[zfs-replica] se le permite RECIBIR en el pool de réplica

```
zfs allow repl create,receive,mount,userprop,destroy,rollback backup
```

En zfs-primary se genera la clave SSH y se copia a zfs-replica para que la conexión funcione sin contraseña:

[zfs-primary]

```
su - repl -c 'ssh-keygen -t ed25519 -N "" -f ~/.ssh/id_ed25519'
```

```
su - repl -c 'ssh-copy-id repl@192.168.100.11'
```

Para comprobarlo:

Desde zfs-primary, `su - repl -c 'ssh repl@192.168.100.11'` debe conectar sin pedir contraseña.

9.3. Replicación incremental con zfs send/receive

La replicación usa `zfs send/receive`, el mecanismo nativo de ZFS para transferir snapshots entre servidores a través de SSH. El primer envío es completo; los siguientes son incrementales gracias al flag `-i`, que indica a ZFS que solo envíe los bloques que han cambiado entre el snapshot anterior y el nuevo. Esto hace que, a partir del primer envío, la réplica sea muy rápida y eficiente.

El script `zfs-replicar.sh` gestiona automáticamente esta lógica: comprueba los snapshots existentes en cada dataset, decide si el envío debe ser completo o incremental, y registra cada operación en `/var/log/zfs-replicar.log`. Se ejecuta como usuario `repl`, 10 minutos después del snapshot diario:

```
10 0 * * * repl /usr/local/bin/zfs-replicar.sh
```

9.4. Funcionamiento interno de la replicación

Para entender qué hace el script por debajo, estos son los comandos equivalentes ejecutados a mano. Primer envío completo:

```
# [zfs-primary] primer envío COMPLETO
```

```
zfs snapshot tank/nfs@base
```

```
zfs send tank/nfs@base | ssh repl@192.168.100.11 zfs receive -F backup/nfs
```

Envío incremental (solo el delta entre dos snapshots):

```
# después, INCREMENTAL (solo el delta entre @base y @nuevo)
```

```
zfs snapshot tank/nfs@nuevo
```

```
zfs send -i tank/nfs@base tank/nfs@nuevo | ssh repl@192.168.100.11 zfs receive backup/nfs
```

Gracias al copy-on-write, ZFS sabe exactamente qué bloques son nuevos y cuáles no han cambiado, por lo que el envío incremental es mucho más rápido que el primero.

9.5. Verificación final

En zfs-replica, `zfs list` muestra el pool backup con los datasets backup/nfs, backup/samba y backup/iscsi-lun0, y `zfs list -t snapshot` muestra los mismos snapshots que en zfs-primary, confirmando que la réplica está sincronizada.

Cuidado:

El snapshot usado como punto de partida del incremental debe existir en ambos servidores. Si se destruye en zfs-primary antes de replicarlo, el siguiente envío incremental fallará y habrá que hacer un envío completo de nuevo.

Si zfs-primary falla (disco roto, sistema operativo corrupto, etc.), hay dos opciones.

Opción 1 → Reparar zfs-primary y restaurar desde zfs-replica. Si se logra arreglar, se usan los datos guardados en la réplica para recuperar todo, enviándolos de vuelta:

en zfs-replica, enviar los datos de vuelta a zfs-primary

```
zfs send -R backup | ssh root@192.168.100.10 zfs receive -F tank
```

Opción 2 → Convertir zfs-replica en servidor principal. Si zfs-primary está roto sin remedio, basta con activar los servicios en zfs-replica: se instalan (`apt install nfs-kernel-server samba targetcli-fb`), se configuran apuntando a backup/nfs, backup/samba y backup/iscsi-lun0, y los clientes pasan a conectarse a zfs-replica. Los datos ya están todos ahí porque se replicaron cada día.

10. Fase 5 — Pruebas y simulación de fallos

Qué se consigue:

Demostrar que todo funciona: los tres accesos desde los clientes y, sobre todo, la recuperación ante un fallo de disco y ante la pérdida de un fichero.

10.1. Probar NFS

```
# [cliente-linux]
apt install -y nfs-common
mkdir -p /mnt/nfs
mount -t nfs zfs-primary:/tank/nfs /mnt/nfs # se monta la carpeta remota
echo "prueba NFS" > /mnt/nfs/prueba.txt    # se escribe en ella

# montaje automático al arrancar, en /etc/fstab:
# zfs-primary:/tank/nfs /mnt/nfs nfs defaults,_netdev 0 0
```

El `_netdev` indica al sistema que espere a tener red antes de montar.

10.2. Probar Samba

```
# [cliente-linux]
apt install -y cifs-utils
mkdir -p /mnt/samba
printf 'username=smbuser\npassword=TU_CONTRASENA\n' > /root/.smbcred
chmod 600 /root/.smbcred    # solo root puede leerlo
mount -t cifs //zfs-primary/datos /mnt/samba \
-o credentials=/root/.smbcred,uid=1000,gid=1000,ioccharset=utf8
```

Se usa un fichero de credenciales para no dejar la contraseña en el historial. Desde Windows, en el Explorador, se escribe `\\192.168.100.10\datos`.

10.3. Probar iSCSI

Aquí se ve lo de «el cliente formatea el disco»: tras conectarse aparece un disco nuevo y es el propio cliente quien lo formatea.

[cliente-linux]

```
apt install -y open-iscsi
```

```
echo "InitiatorName=iqn.2026-01.local.cliente-linux:init0" >
/etc/iscsi/initiatorname.iscsi
```

```
systemctl restart iscsid
```

```
iscsiadm -m discovery -t sendtargets -p 192.168.100.10 # descubre los targets
```

```
iscsiadm -m node -T iqn.2026-01.local.zfs-primary:target0 -p 192.168.100.10
--login
```

```
lsblk # aparece un disco nuevo, p.ej. /dev/sdb
```

```
mkfs.ext4 /dev/sdb # EL CLIENTE lo formatea
```

```
mkdir -p /mnt/iscsi && mount /dev/sdb /mnt/iscsi
```


10.4. Simular un fallo de disco (La demostración estrella)

Como el pool está en mirror, se puede «romper» un disco y el servidor sigue dando servicio:

```
# [zfs-primary]
zpool offline tank /dev/disk/by-id/ata-VBOX_HARDDISK_VBbbbbbbbbb # disco caído
zpool status tank          # el pool aparece DEGRADED pero los datos siguen
                             accesibles
zpool replace tank ...disco_viejo... ...disco_nuevo...          # se sustituye
zpool status tank          # se ve el 'resilver': ZFS reconstruye el espejo
```

10.5. Recuperar datos con un snapshot

```
# [zfs-primary]
rm /tank/nfs/prueba.txt
zfs list -t snapshot tank/nfs # se busca un snapshot anterior al borrado
zfs rollback tank/nfs@<snapshot>
ls /tank/nfs/                  # el fichero vuelve a estar
```

10.6. Verificar la integridad del pool

```
# [zfs-primary]
zpool scrub tank      # ZFS lee todo y comprueba los checksums
zpool status tank     # muestra si ha encontrado y corregido algún error
```

11. Uso de contraseñas y usuarios

A lo largo del proyecto intervienen varias cuentas, y conviene tenerlas recogidas en un único sitio porque no todas son del mismo tipo ni se gestionan igual. La tabla resume cada usuario, dónde existe y para qué sirve.

Usuario	Dónde	Tipo de cuenta	Credencial	Para qué sirve
admin (sudo)	todas las máquinas	cuenta del sistema	contraseña personal	administración general, instalar paquetes
root	servidores	cuenta del sistema	se usa vía sudo	ejecutar la configuración de los servicios
smbuser	zfs-primar y	solo Samba (nologin)	abc123.	acceso al recurso [datos] por SMB
repl	primary y replica	sistema (sin login interactivo)	clave SSH ed25519 (sin contraseña)	enviar y recibir la réplica

Hay dos detalles que conviene destacar. El primero es que la contraseña de Samba (por ejemplo abc123. para el usuario smbuser) es independiente de la del sistema: se guarda en la base de datos propia de Samba y por eso se crea con smbpasswd, no con passwd. El usuario smbuser se crea además con nologin, de modo que sirve para compartir ficheros pero no puede iniciar sesión en el equipo.

El segundo es que el usuario repl no usa contraseña en absoluto: se autentica con una clave SSH ed25519 y solo tiene los permisos de ZFS estrictamente necesarios (enviar en el primario, recibir en la réplica). Es una cuenta de servicio, no una cuenta de persona.

12. Presupuesto

A continuación se detalla el presupuesto estimado del proyecto, separando el hardware y las licencias por un lado, y la mano de obra por otro. Sobre el subtotal se aplica un 10 % de contingencia y, finalmente, el IVA.

12.1. Hardware y licencias

Concepto	Cantidad	Precio unitario (€)	Total (€)
Servidor principal (16 GB RAM)	1	500	500
Servidor secundario (16 GB RAM)	1	500	500
SSD 500 GB para SO	2	40	80
HDD 2 TB para almacenamiento ZFS	4	60	240
Switch Gigabit 8 puertos	1	30	30
Cableado de red	1	20	20
Licencia Windows 11 Pro (cliente)	1	150	150
Subtotal hardware y licencias			1.520

12.2. Mano de obra

Tarea	Horas	Precio/hora (€)	Total (€)
Instalación y preparación de servidores	4	35	140
Configuración de ZFS y almacenamiento	5	35	175

Tarea	Horas	Precio/hora (€)	Total (€)
Configuración de NFS	2	35	70
Configuración de Samba	3	35	105
Configuración de iSCSI	3	35	105
Configuración de replicación (snapshots y SSH)	3	35	105
Pruebas, validación y documentación	4	35	140
Subtotal mano de obra			840

12.3. Resumen

Concepto	Importe (€)
Hardware y licencias	1.520
Software	0
Mano de obra	840
Subtotal	2.360
Contingencia (10 %)	236
Base imponible	2.596
IVA (21 %)	545,16
TOTAL PRESUPUESTADO	3.141,16

13. Mejoras a futuro

13.1. Aplicación web para una gestión sencilla

La principal mejora prevista es una aplicación web que sirva de panel de control para administrar todo el almacenamiento sin tocar la terminal. La idea es que, desde un navegador, se puedan hacer las tareas más habituales del día a día:

- Ver de un vistazo el estado de los pools (ONLINE, DEGRADED), el espacio usado y libre, y si la última réplica se completó bien.
- Crear y restaurar snapshots con un par de clics, sin recordar la sintaxis de `zfs snapshot` o `zfs rollback`.
- Gestionar los recursos compartidos: dar de alta usuarios de Samba, crear o quitar exportaciones de NFS y revisar quién tiene acceso.
- Lanzar o programar la replicación y consultar el registro de `/var/log/zfs-replicar.log` desde la propia interfaz.
- Recibir avisos cuando un disco falle o cuando un scrub encuentre errores.

En cuanto a la forma de construirla, la opción más rápida sería apoyarse en un panel ya existente como Cockpit (con su módulo de ZFS) o Webmin, que aportan buena parte de la monitorización sin programar casi nada. La alternativa más flexible sería un panel propio, por ejemplo en Python con Flask, que ejecute por debajo los mismos comandos `zpool` y `zfs` que ya se usan en este proyecto y muestre los resultados de forma amigable.

13.2. Por qué se eligió esta mejora

Se eligió esta mejora porque todo el montaje actual es de línea de comandos, lo que exige conocer ZFS y Linux para operarlo. En el escenario al que apunta el presupuesto —una pequeña empresa— no siempre hay personal con ese nivel técnico, y las tareas más frecuentes (recuperar un fichero borrado, dar de alta a un usuario, comprobar que la copia se hizo) son precisamente las que más se beneficiarían de un botón en lugar de un comando.

Una interfaz web baja la barrera de entrada, reduce el riesgo de errores al teclear comandos delicados y permite delegar el mantenimiento rutinario sin dar acceso completo al servidor. Además, es una evolución natural del proyecto: no cambia la base (sigue siendo ZFS con sus snapshots y su réplica), solo le pone una capa de gestión por encima.

14. Conclusiones y dificultades encontradas

Una vez terminado el proyecto, queda un servidor que ofrece los tres protocolos a la vez sobre el mismo almacenamiento ZFS, con snapshots automáticos y una réplica en un segundo servidor (los dos con pool en mirror). Se ha verificado que funciona escribiendo desde los clientes, recuperando un borrado con un snapshot, aguantando el fallo de un disco y comprobando que los datos llegan a la réplica.

Las principales dificultades encontradas y cómo se resolvieron:

- **ZFS y DKMS.**

Una lección clave fue entender que ZFS no viene en el núcleo por motivos de licencia y que DKMS lo recompila; conviene mantener las linux-headers al día.

- **IQN de iSCSI.**

Se invirtió bastante tiempo en un fallo de login hasta descubrir que el IQN del cliente debía ser exactamente igual al de la ACL del servidor.

Permisos de Samba. El bit setgid (2770) fue determinante para que los ficheros nuevos heredaran el grupo.

15. Scripts de configuración

Para automatizar el montaje se prepararon scripts separados: tres para los servicios de la Fase 3 (uno por servicio) y tres para los snapshots y la replicación de la Fase 4. Todos empiezan con `set -euo pipefail` para parar al primer error y comprueban por su cuenta que existe lo que necesitan antes de tocar nada.

15.1. setup-nfs.sh

```
#!/usr/bin/env bash
# setup-nfs.sh - Configura el servicio NFS sobre ZFS.
# Ejecutar en zfs-primary como root: sudo ./setup-nfs.sh
set -euo pipefail

DATASET="tank/nfs"
EXPORT_DIR="/tank/nfs"
NET="192.168.100.0/24"
OPTS="rw, sync, no_subtree_check, root_squash"

echo ">> 1/4 Instalando el servidor NFS..."
apt update && apt install -y nfs-kernel-server

echo ">> 2/4 Comprobando que existe el dataset $DATASET..."
zfs list "$DATASET" >/dev/null 2>&1 || zfs create "$DATASET"

echo ">> 3/4 Anadiendo la exportacion a /etc/exports (si no existe ya)..."
LINE="$EXPORT_DIR $NET($OPTS)"
grep -qxF "$LINE" /etc/exports 2>/dev/null || echo "$LINE" >> /etc/exports

echo ">> 4/4 Aplicando y activando el servicio..."
exportfs -ra
systemctl enable --now nfs-kernel-server
exportfs -v
```

15.2. setup-iscsi.sh

```
#!/usr/bin/env bash
# setup-iscsi.sh - Configura el target iSCSI sobre un zvol de ZFS.
# Ejecutar en zfs-primary como root (primera instalacion): sudo
./setup-iscsi.sh
set -euo pipefail

ZVOL="tank/iscsi-lun0"; ZVOL_SIZE="3G"; ZVOL_DEV="/dev/zvol/tank/iscsi-lun0"
TARGET_IQN="iqn.2026-01.local.zfs-primary:target0"
INITIATOR_IQN="iqn.2026-01.local.cliente-linux:init0"
PORTAL_IP="192.168.100.10"

apt update && apt install -y targetcli-fb
zfs list "$ZVOL" >/dev/null 2>&1 || \
  zfs create -V "$ZVOL_SIZE" -o volblocksize=16k "$ZVOL"

targetcli /backstores/block create name=lun0 dev="$ZVOL_DEV"
targetcli /iscsi create "$TARGET_IQN"
targetcli /iscsi/"$TARGET_IQN"/tpg1/luns create /backstores/block/lun0
targetcli /iscsi/"$TARGET_IQN"/tpg1/acls create "$INITIATOR_IQN"
targetcli /iscsi/"$TARGET_IQN"/tpg1/portals delete 0.0.0.0 3260 || true
targetcli /iscsi/"$TARGET_IQN"/tpg1/portals create "$PORTAL_IP"
targetcli saveconfig
systemctl enable --now rtslib-fb-targetctl.service
targetcli ls /iscsi
```


15.3. setup-samba.sh

```
#!/usr/bin/env bash
# setup-samba.sh - Configura el servicio Samba sobre ZFS.
# Ejecutar en zfs-primary como root: sudo ./setup-samba.sh
set -euo pipefail

DATASET="tank/samba"; SHARE_DIR="/tank/samba"; SHARE_NAME="datos"
SMB_USER="smbuser"; SMB_GROUP="smbusers"

apt update && apt install -y samba
zfs list "$DATASET" >/dev/null 2>&1 || zfs create "$DATASET"
getent group "$SMB_GROUP" >/dev/null || groupadd "$SMB_GROUP"
id "$SMB_USER" >/dev/null 2>&1 || \
    useradd -M -s /usr/sbin/nologin -G "$SMB_GROUP" "$SMB_USER"

echo ">> Defina la contraseña de Samba para '$SMB_USER':"
smbpasswd -a "$SMB_USER"; smbpasswd -e "$SMB_USER"

chgrp -R "$SMB_GROUP" "$SHARE_DIR"; chmod 2770 "$SHARE_DIR"

if ! grep -q "^\[${SHARE_NAME}\]" /etc/samba/smb.conf; then
cat >> /etc/samba/smb.conf <<EOF
[${SHARE_NAME}]
    comment = Comparticion sobre ZFS
    path = $SHARE_DIR
    browseable = yes
    read only = no
    guest ok = no
    valid users = @$SMB_GROUP
    force group = $SMB_GROUP
    create mask = 0660
    directory mask = 2770
EOF
fi
testparm -s
systemctl restart smbd nmbd; systemctl enable smbd nmbd
```

15.4. zfs-snapshot.sh

```
#!/usr/bin/env bash
# zfs-snapshot.sh - Toma un snapshot con fecha y hora de cada dataset de
# tank.
# Se ejecuta a diario desde cron (00:00).
set -euo pipefail

FECHA="$(date +%Y-%m-%d_%H-%M)"
DATASETS="tank/nfs tank/samba tank/iscsi-lun0"

for ds in $DATASETS; do
    zfs snapshot "${ds}@${FECHA}"
done
```

15.5. zfs-cleanup.sh

```
#!/usr/bin/env bash
# zfs-cleanup.sh - Conserva solo los 7 snapshots mas recientes de cada
# dataset.
# Se ejecuta 5 minutos despues del snapshot diario.
set -euo pipefail

CONSERVAR=7
DATASETS="tank/nfs tank/samba tank/iscsi-lun0"

for ds in $DATASETS; do
    zfs list -H -t snapshot -o name -s creation -d 1 "$ds" \
    | head -n -"$CONSERVAR" \
    | while read -r snap; do
        zfs destroy "$snap"
    done
done
```

15.6. zfs-replicar.sh

```
#!/usr/bin/env bash
# zfs-replicar.sh - Replica los datasets de tank a backup (zfs-replica) por
SSH.
# Decide automaticamente si el envio es completo o incremental.
# Se ejecuta como usuario 'repl', 10 minutos despues del snapshot diario.
set -euo pipefail

DESTINO="repl@192.168.100.11"
LOG="/var/log/zfs-replicar.log"
DATASETS="nfs samba iscsi-lun0"

log() { echo "$(date '+%Y-%m-%d %H:%M:%S') $*' >> "$LOG"; }

for ds in $DATASETS; do
    ORIGEN="tank/$ds"; REMOTO="backup/$ds"
    NUEVO="$(zfs list -H -t snapshot -o name -s creation -d 1 "$ORIGEN" | tail
-n 1)"
    [ -z "$NUEVO" ] && { log "AVISO: $ORIGEN sin snapshots"; continue; }

    if ssh "$DESTINO" zfs list "$REMOTO" >/dev/null 2>&1; then
        BASE="$(ssh "$DESTINO" zfs list -H -t snapshot -o name -s creation -d 1
"$REMOTO" | tail -n 1)"
        BASE_NOMBRE="${BASE##*@}"
        if zfs list "${ORIGEN}@${BASE_NOMBRE}" >/dev/null 2>&1; then
            log "Incremental $ORIGEN: @${BASE_NOMBRE} -> ${NUEVO##*@}"
            zfs send -i "${ORIGEN}@${BASE_NOMBRE}" "$NUEVO" | ssh "$DESTINO" zfs
receive "$REMOTO"
        else
            log "ERROR: la base @${BASE_NOMBRE} no existe en origen; hace falta
envio completo"
        fi
    else
        log "Envio completo inicial $ORIGEN -> $REMOTO (${NUEVO##*@})"
        zfs send "$NUEVO" | ssh "$DESTINO" zfs receive -F "$REMOTO"
    fi
done
log "Replicacion terminada."
```

16. Bibliografía

Las fuentes consultadas durante el proyecto, todas documentación oficial o de referencia:

- OpenZFS Documentation. Documentación oficial del proyecto OpenZFS (instalación, administración, snapshots y replicación). <https://openzfs.github.io/openzfs-docs/>
- Debian Wiki - ZFS. Instalación de ZFS en Debian mediante el componente contrib y DKMS. <https://wiki.debian.org/ZFS>
- Páginas de manual de ZFS: zpool, zfs, zpool-create, zfs-allow, zfs-send y zfs-receive.
- Documentación de Samba. Manual de smb.conf y guías oficiales del proyecto. <https://www.samba.org/samba/docs/>
- NFS en Linux. Página de manual exports y documentación de nfs-kernel-server.
- Linux-IO (LIO) / targetcli. Documentación del target iSCSI del núcleo de Linux y del proyecto open-iscsi para el iniciador. <https://github.com/open-iscsi/open-iscsi>
- Debian Administrator's Handbook. Referencia general de administración de Debian 13. <https://debian-handbook.info/>